

Технический справочник

TDS Toolkit

Написание декодировщика протокола

Для цифровых осциллографов серий TDS 500, TDS 600 и TDS 700

Документ TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Лицензия MIT.

TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Все права защищены.

В этом документе описан интерфейс подключаемых декодировщиков TDS Toolkit. Информация в этом издании заменяет все ранее опубликованные материалы.

TDS Toolkit — это хост-программа; она не является продуктом Tektronix. Названия Tektronix и TDS используются лишь для обозначения приборов, которые поддерживает программа.

Данное программное обеспечение выпущено под лицензией MIT. ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ «КАК ЕСТЬ», БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, ЯВНЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ. Полный текст сопровождает программу под именем LICENSE.

Содержание

Сопутствующие руководства	iv
1 Модель декодировщика	1-1
Что такое декодировщик	1-1
Один файл, один протокол	1-1
Контракт	1-1
Обнаружение	1-1
Где находятся файлы	1-1
2 Тип Signal	2-1
Что получает декодирование	2-1
Отсчёты и время	2-1
Уровни, а не отсчёты	2-1
Откуда берётся Signal	2-2
3 Контракт декодировщика	3-1
Атрибуты класса	3-1
Четыре атрибута	3-1
Роли и источники	3-1
Методы	3-2
decode	3-2
Параметры	3-3
Тип Param	3-3
4 Кадры и отрисовка	4-1
Тип Frame	4-1
Одно декодированное событие	4-1
Состояние	4-1
Отрисовка результата	4-2
Четыре небольших метода	4-2
Цвет	4-2

5 Построение декодировщика	5-1
Разобранный пример	5-1
Протокол	5-1
Шаги	5-1
Код	5-1
Импорты и атрибуты	5-1
Восстановление битов	5-2
Сбор слов	5-2
Отрисовка	5-3
Короткий путь для последовательных протоколов	5-4
Не реализуйте выборку битов заново	5-4
6 Установка и тестирование	6-1
Установка	6-1
Поместите и откройте	6-1
Тестирование	6-1
Подайте на него синтетический Signal	6-1
Тестируйте каждый декодировщик	6-2
Контрольный список	6-2
Прежде чем полагаться на него	6-2

Список таблиц

Таблица 1–1: Файлы подсистемы декодирования	1–2
Таблица 2–1: Элементы типа Signal	2–2
Таблица 3–1: Атрибуты класса Decoder	3–1
Таблица 3–2: Методы декодировщика	3–2
Таблица 3–3: Виды Param	3–3
Таблица 4–1: Поля типа Frame	4–1
Таблица 4–2: Биты состояния кадра	4–2

Предисловие

В этом руководстве описано, как добавить декодировщик протокола в TDS Toolkit. Декодировщик — это один файл на Python. Он читает захваченную осциллограмму и возвращает список декодированных событий; программа перечисляет его, сопоставляет его входы, запускает его и рисует результат без какого-либо дополнительного кода.

Предполагается, что читатель — практикующий программист, свободно владеющий Python. Знание остальной части программы не требуется: весь интерфейс — это один базовый класс и три небольших типа данных, все в модуле `tds_decode.py`.

Читайте это руководство рядом с двумя файлами, поставляемыми с программой: `decoders/_template.py` — заготовкой для копирования, и `decoders/README.txt` — кратким изложением того, что следует далее.

Сопутствующие руководства

Интерфейс и файлы, которые следует читать рядом с этим руководством:

- `decoders/_template.py` — заготовка для копирования.
- `decoders/README.txt` — краткое изложение того, что следует далее.
- `tds_decode.py` — движок: базовый класс `Decoder` и типы `Signal`, `Frame` и `Param`.

Модель декодировщика

Что такое декодировщик

Один файл, один протокол	Декодировщик — это отдельный файл на Python в папке decoders, определяющий один подкласс Decoder. Каждый файл — это один протокол. Вкладка «Осциллограммы» перечисляет каждый найденный декодировщик, поэтому для добавления протокола достаточно одного файла, а остальная программа не меняется. Декодирование выполняется целиком по захваченной осциллограмме, которую вкладка «Осциллограммы» уже содержит: по каналу, прочитанному из прибора, или по сохранённой опорной памяти, загруженной из файла. Ничто в декодировщике не обращается к прибору. В этом и состоит смысл конструкции. Такой подход работает на любом TDS, включая базовые модели, прошивка которых не может запустить собственное приложение декодирования шины, потому что декодирование происходит в хосте, по отсчётам, после захвата.
Контракт	Весь интерфейс — это базовый класс Decoder. Декодировщик задаёт четыре атрибута класса и реализует один метод. Остальные три метода имеют рабочие значения по умолчанию и переопределяются лишь для отображения собственных полей протокола в таблице и на графике. Три типа данных, с которыми работает декодировщик, — Signal, Frame и Param — описаны в следующих главах. При запуске и всякий раз при открытии вкладки «Осциллограммы» программа просматривает папку decoders на предмет файлов *.py, импортирует каждый по отдельности и создаёт экземпляр каждого найденного подкласса Decoder. Файл, имя которого начинается с подчёркивания, считается вспомогательным и пропускается — именно так _template.py остаётся заготовкой и никогда не появляется в списке как пустой протокол.
Обнаружение	Импорт выполняется с защитой от ошибок. Файл, который не импортируется — из-за синтаксической ошибки или неверной ссылки, — пропускается с записью в журнал программы, и не выводит из строя весь список. Незавершённый декодировщик никогда не скрывает работающие. Когда две папки предлагают декодировщик с одинаковым NAME, приоритет имеет папка, просматриваемая позже, поэтому локально написанная копия переопределяет поставляемую.

Где находятся файлы

Декодировщики, поставляемые с программой, находятся в папке `decoders` внутри установки. Копия, доступная для записи, хранится рядом с файлом настроек `tdstoolkit.json` и пополняется из поставляемого набора при каждом запуске, точно так же, как библиотека масок. Декодировщик, добавленный в более поздней сборке, копируется без перезаписи локально написанного; удалённый поставляемый декодировщик возвращается при следующем запуске, поскольку поставляемый набор должен присутствовать. Поместите новый файл в эту папку, и он появится в списке при следующем открытии вкладки. Пересборка не требуется.

Таблица 1–1: Файлы подсистемы декодирования

Файл	Роль
<code>tds_decode.py</code>	Движок: базовый класс <code>Decoder</code> , типы <code>Signal</code> , <code>Frame</code> и <code>Param</code> , константы состояния, общий помощник для последовательных протоколов и обнаружение. Декодировщик импортирует из него и не изменяет его.
<code>decoders/_template.py</code>	Заготовка. Скопируйте её, чтобы начать новый декодировщик.
<code>decoders/README.txt</code>	Краткие инструкции и карта примеров декодировщиков.
<code>decoders/.py</code>	По одному протоколу в каждом: <code>uart.py</code> , <code>i2c.py</code> , <code>spi.py</code> , <code>can.py</code> и остальные. Читайте тот, что ближе всего к добавляемому протоколу.

Тип Signal

Что получает декодирование

	Методу <code>decode()</code> передаётся по одному <code>Signal</code> на каждый вход, нужный протоколу. <code>Signal</code> — это одна захваченная запись, сведённая к тому, что нужно декодировщику: целочисленные отсчёты и сведения о времени, превращающие индекс отсчёта во время. Он не содержит ни вольт, ни масштабирования отображения. Декодировщик сравнивает отсчёт с порогом и ему никогда не нужно знать напряжение, поэтому необработанные отсчёты оцифровщика передаются без масштабирования, и запись в четверть миллиона точек ничего не стоит передать.
Отсчёты и время	Порог принятия решения, мёртвая зона гистерезиса и размах сигнала вычисляются до вызова <code>decode()</code>. Декодировщик не вычисляет собственный порог. Порог — это середина между крайними значениями записи; гистерезис — восьмая часть размаха, достаточно широкая, чтобы отбросить звон фронта, но не настолько, чтобы медленный фронт его не пересёк. Эти значения читаются, а не задаются.
Уровни, а не отсчёты	Для большинства протоколов полезная форма записи — не необработанные отсчёты, а чистый логический уровень на каждый отсчёт. Метод <code>digitize()</code> возвращает именно это: 0 или 1 для каждого отсчёта, с применённым собственным гистерезисом записи, так что звон фронта не читается как несколько пересечений. Вызовите его один раз и пройдите по результату. Протокол, которому нужны необработанные отсчёты, например биполярная линия с двумя порогами, вместо этого читает <code>sig.data</code> напрямую.

Таблица 2–1: Элементы типа Signal

Элемент	Назначение
sig.data	Целочисленные отсчёты — индексируемая последовательность необработанных отсчётов оцифровщика.
sig.n	Число отсчётов, то же, что len(sig.data).
sig.dt	Секунд на отсчёт.
sig.t0	Время первого отсчёта, в секундах относительно запуска.
sig.first_index	Смещение первого отсчёта в записи. Прибавьте его к локальному индексу отсчёта для получения абсолютной позиции в записи.
sig.threshold	Уровень принятия решения, в отсчётах. Задаётся каркасом.
sig.hysteresis	Мёртвая зона вокруг порога, в отсчётах. Задаётся каркасом.
sig.span	Разность между наибольшим и наименьшим отсчётом записи. Задаётся каркасом; размах около нуля означает плоский источник без сигнала.
sig.time_at(i)	Время в секундах при локальном индексе отсчёта i, то есть $t0 + i \times dt$.
sig.digitize()	Новый bytearray с одним уровнем 0/1 на каждый отсчёт, с гистерезисом.
sig.auto_threshold()	Пересчитывает порог, гистерезис и размах. Уже вызывается каркасом; вызывайте его только для Signal, построенного вручную (см. раздел 6).

ПРИМЕЧАНИЕ. Signal содержит необработанные отсчёты, а не вольты. Сравнивайте с sig.threshold, а не с напряжением. Декодировщик, работающий правильно при одном вертикальном масштабе, будет работать правильно и при любом другом, потому что порог смещается вместе с записью.

Откуда берётся Signal

Signal, который получает decode(), строится программой, а не декодировщиком, из захваченной осциллограммы методом класса Signal.from_waveform(). Он читает целочисленные уровни, интервал между отсчётами и смещение запуска из преамбулы осциллограммы, конструирует Signal и вызывает для него auto_threshold(), прежде чем передать его декодировщику. Signal строится вручную только при написании теста, что рассматривается в разделе 6.

Контракт декодировщика

Атрибуты класса

Подкласс декодировщика задаёт четыре атрибута класса. NAME и DESCRIPTION именуют его в списке протоколов. SOURCES объявляет входные осциллограммы, которые ему нужны. PARAMS объявляет настройки, предлагаемые вкладкой. Всё остальное наследуется.

Таблица 3–1: Атрибуты класса Decoder

Атрибут	Назначение
NAME	Имя протокола. Отображается в списке и используется как заголовок метки на графике. Оставьте его пустым или равным "?", и загрузчик не включит класс в список; именно так заготовка не попадает в список.
DESCRIPTION	Одна строка для второго столбца списка.
SOURCES	Список имён ролей по порядку. Одна роль — это однопроводный протокол; если ролей две или более, каждой сопоставляется источник. Завершающий "?" отмечает необязательную роль, которую можно оставить несопоставленной.
PARAMS	Список объектов Param, по одному элементу управления на каждый. Может быть пустым.
COLOUR	Цвет шеврона для исправного кадра, в виде "#rrggbb". Цвета задаёт сам декодировщик; в программе нет настройки цвета для декодировщиков. Задайте его, чтобы весь протокол имел один цвет.
FAULT_COLOUR	Цвет шеврона для сбойного кадра. По умолчанию красный, чтобы ошибка выделялась при любом цвете протокола.

Роли и источники

SOURCES — это и количество осциллограмм, нужных протоколу, и их имена. Однопроводный протокол объявляет одну роль, по соглашению "Line". Двухпроводный протокол, такой как I2C, объявляет ["SDA", "SCL"]; вкладка показывает по одному селектору источника на роль, и decode() получает Signal для каждой. Завершающий вопросительный знак, как в ["CLK", "MOSI", "MISO?", "CS?"] у SPI, отмечает роль, которую оператор может оставить несопоставленной; её Signal тогда отсутствует в словаре, который получает decode(), и код decode() проверяет его наличие.

Методы

decode `decode(self, signals, params)` — единственный обязательный метод. `signals` — это словарь, сопоставляющий каждое имя роли, без завершающего "?", её `Signal`; необязательная роль, которая не была сопоставлена, отсутствует. `params` — это словарь, сопоставляющий каждый ключ `Param` значению, выбранному оператором. Метод возвращает кортеж из двух элементов: список объектов `Frame` и однострочное примечание.

Примечание отображается над таблицей результатов. Используйте его, чтобы сообщить, что вывело декодирование, например обнаруженную скорость передачи, число отсчётов на бит и число найденных кадров, или, когда запись не удаётся прочитать, чтобы объяснить причину. При неудаче верните пустой список кадров и примечание, объясняющее проблему одной фразой.

Таблица 3-2: Методы декодировщика

Метод	Возвращает
<code>decode(signals, params)</code>	<code>(frames, note)</code> . Само декодирование. Обязательный.
<code>columns()</code>	Заголовки таблицы после столбца «Время» в виде списка пар <code>(heading, width_in_pixels)</code> . Переопределите, чтобы назвать поля протокола.
<code>row(frame, ascii)</code>	Ячейки таблицы для одного кадра, соответствующие <code>columns()</code> после времени. Переопределите, чтобы форматировать поля протокола.
<code>label(frame, ascii)</code>	Самая короткая полезная строка для шеврона на графике. Переопределите, чтобы она была лаконичной.
<code>colour(frame)</code>	Цвет шеврона для одного кадра. По умолчанию <code>COLOUR</code> для исправного кадра и <code>FAULT_COLOUR</code> для сбойного. Переопределите, чтобы задавать цвет для каждого шеврона, с ветвлением по <code>frame.kind</code> .

Аргумент `ascii`, передаваемый в `row()` и `label()`, — это состояние переключателя `Hex / ASCII` на вкладке. Учитывайте его там, где это имеет смысл — байт, показанный как символ или как две шестнадцатеричные цифры, — и игнорируйте там, где не имеет.

Параметры

Каждый `Param` в списке `PARAMS` становится одним элементом управления на вкладке, поэтому протокол добавляет собственные параметры без какого-либо кода пользовательского интерфейса. Конструктор — `Param(key, label, kind, choices, default, lo, hi, unit, note)`. `key` — это ключ словаря, который читает `decode()`; `label` — это подпись, которую видит оператор; `kind` — это одна из трёх строк.

Таблица 3-3: Виды `Param`

kind	Элемент управления и поля
"choice"	Раскрывающийся список. <code>choices</code> — это список пар (<code>value</code> , <code>label</code>); <code>value</code> — это то, что попадает в <code>params</code> , а <code>label</code> — то, что показывается. <code>default</code> — одно из значений.
"int"	Ввод целого числа. <code>lo</code> и <code>hi</code> ограничивают его; <code>unit</code> — это суффикс подписи, например "bits". Значение <code>default</code> , равное 0, по соглашению означает "auto", о чём должно сообщать <code>note</code> .
"bool"	Флажок. <code>default</code> — это <code>True</code> или <code>False</code> .

Тип `Param`

Если `default` опущен, он принимает первое значение для `choice`, `False` для `bool` и 0 для `int`. Необязательное `note` показывается рядом с элементом управления как подсказка.

Кадры и отрисовка

Тип Frame

Одно декодированное событие

Frame — это одно декодированное событие: символ UART, байт I2C, слово SPI, состояние шины. Тип плоский и общий для всех протоколов. Он содержит первичные данные, необязательные вторичные, длину, состояние (status) и вид (kind). Смысл полей определяет сам декодировщик, поскольку кадр отрисовывает только тот декодировщик, который его создал, но их форма фиксирована.

Создавайте Frame с помощью именованных аргументов; каждое поле имеет значение по умолчанию. Два индексных поля важнее всего и в них легче всего ошибиться. `index` и `end_index` — это абсолютные позиции отсчётов в записи: график рисует прямоугольник шеврона от одного до другого. Формируйте их как `sig.first_index` плюс локальный индекс отсчёта, ровно так же, как время получается из `sig.time_at()` от локального индекса.

Таблица 4-1: Поля типа Frame

Поле	Назначение
<code>time</code>	Время начала события в секундах, из <code>sig.time_at(local_index)</code> .
<code>index</code>	Абсолютный начальный отсчёт: <code>sig.first_index</code> + локальное начало.
<code>end_index</code>	Абсолютный конечный отсчёт: <code>sig.first_index</code> + локальный конец. Прямоугольник занимает диапазон от <code>index</code> до <code>end_index</code> .
<code>value</code>	Первичные данные: байт, слово, измеренное значение.
<code>tag</code>	Необязательные вторичные данные: адрес I2C, слово MISO у SPI, измеренная частота. 0, если не используется.
<code>length</code>	Длина данных в тех единицах, которыми считает протокол. По умолчанию 1.
<code>status</code>	Побитовое ИЛИ констант состояния; ОК (ноль), когда кадр исправен.
<code>kind</code>	Произвольная строка, например "byte", "addr", "data", "start" или "stop", по которой <code>row()</code> и <code>label()</code> ветвятся, чтобы отрисовать каждую часть кадра по-своему.

Состояние status — это битовая маска. Исправный кадр — это ОК, то есть ноль. Кадр с проблемой объединяет по ИЛИ одну или несколько констант из таблицы 4-2. Каркас рисует любой кадр с ненулевым status как ошибку — красный шеврон и ведущий восклицательный знак в метке, — при этом декодировщик не делает ничего, кроме установки бита.

Таблица 4-2: Биты состояния кадра

Константа	Значение	Назначение
OK	0	Исправен; без ошибки.
FRAMING_ERROR	1	Кадр не завершился так, как требует его формат, например стоп-бит UART, который не был маркой.
PARITY_ERROR	2	Бит чётности не совпал с данными.
TRUNCATED	4	Запись закончилась посреди кадра.
ACK_ERROR	8	Байт, ожидавший подтверждения, не был подтверждён.

Импортируйте используемые константы из `tds_decode` вместе с `Decoder`, `Frame` и `Param`. Не определяйте никаких значений status сверх этих; пять перечисленных выше — это то, что понимает отрисовщик.

Отрисовка результата

Четыре небольших метода

Базовый класс отрисовывает кадр как обычный байт: hex, ASCII и слово состояния. Переопределите `columns()`, `row()` и `label()`, чтобы использовать собственные термины протокола. `columns()` возвращает заголовки после столбца «Время», каждый с шириной в пикселях. `row()` возвращает по одной ячейке на заголовок в том же порядке. `label()` возвращает самую короткую полезную строку для шеврона, рисуемого на самой осциллограмме.

Протокол, кадры которого бывают разных видов, использует в этих методах ветвление по `frame.kind`, чтобы START, адресный байт и байт данных отображались в таблице и на графике каждый по-своему. `label()` должен быть лаконичным: он рисуется внутри небольшого прямоугольника над кривой, поэтому подходящая длина — от одного до трёх символов.

Цвет Четвёртый метод, `colour(frame)`, задаёт цвет шеврона, и декодировщик владеет им полностью: в программе нет настройки цвета для декодировщиков. По умолчанию исправный кадр окрашивается в цвет класса `COLOUR`, а сбойный — в `FAULT_COLOUR`, поэтому установка `COLOUR` придаёт всему протоколу один цвет. Переопределите `colour()` и ветвитесь по `frame.kind`, чтобы задавать цвет для каждого шеврона, так что адрес, данные и обрамление одной линии отображаются каждый своим цветом. Верните `"#rrggbb"`; для видов, которые декодировщик не окрашивает сам, положитесь на базовый класс.

Построение декодировщика

Разобранный пример

Протокол

В этой главе строится полный декодировщик из заготовки. Пример протокола — одна самосинхронизирующаяся линия, передающая один бит на импульс: уровень линии поднимается, остаётся высоким в течение доли периода бита и спадает; короткий высокий уровень — это 0, а длинный высокий — это 1, что определяется тем, превышает ли время высокого уровня половину периода. Биты собираются в слова выбранного размера и порядка. К этому семейству относятся пиксельная линия WS2812 и инфракрасный пульт NEC, и его нет среди встроенных протоколов, поэтому это подлинно новый декодировщик, а не копия существующего.

Шаги

1. *Скопируйте заготовку:* Скопируйте `decoders/_template.py` в `decoders/pulsebits.py`.
2. *Назовите протокол:* Переименуйте класс и задайте `NAME`, `DESCRIPTION` и `SOURCES`.
3. *Объявите настройки:* Заполните `PARAMS` элементами управления размером слова и порядком бит.
4. *Напишите декодирование:* Восстановите биты из дорожки уровней, затем соберите их в слова.
5. *Отрисуйте поля:* Переопределите `columns()`, `row()` и `label()`, чтобы показать слово в шестнадцатеричном и двоичном виде.
6. *Протестируйте его:* Подайте на декодировщик синтетический `Signal` и проверьте результат с помощью `assert`, как показано в разделе 6.

Код

Импорты и атрибуты

Заголовок импортирует только из `tds_decode`, именуется класс и объявляет два параметра, которые предложит вкладка: размер слова и порядок бит. `SOURCES` содержит одну роль, поэтому вкладка показывает единственный селектор источника.

```

from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]

```

Восстановление битов

decode() берёт единственный Signal, защищается от плоского источника и сводит запись к уровню 0/1 на каждый отсчёт с помощью digitize(). Затем он проходит по дорожке уровней. Каждый бит начинается с нарастающего фронта; серия высоких отсчётов — это импульс, следующая за ней серия низких отсчётов тянется до следующего нарастающего фронта, и вместе они составляют период бита. Значение бита определяется сравнением времени высокого уровня с периодом.

```

def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = []          # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]:     # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1

```

Сбор слов

Затем биты берутся группами по `word_bits`, начиная со старшего или младшего значащего, как указывает параметр. Группа, оказавшаяся неполной из-за того, что запись закончилась, всё равно выдаётся, с состоянием, помеченным `TRUNCATED`, чтобы отрисовщик нарисовал её как ошибку. Время каждого `Frame` берётся от первого бита, а сам кадр занимает интервал от начала первого бита до конца последнего. Примечание сообщает итоги.

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

Отрисовка

Три метода отрисовки показывают слово в шестнадцатеричном виде и в двоичном виде его собственной разрядности, а также отмечают усечённое слово. `label()` ограничивается двумя шестнадцатеричными цифрами с ведущим восклицательным знаком при ошибке — этому соглашению следуют встроенные декодировщики.

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "0%db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

Этот файл — полный декодировщик, готовый к включению в список. Помещённый в папку `decoders`, он появляется на вкладке как `"Pulse-width bits"` с полем размера слова и раскрывающимся списком порядка бит и декодирует любой источник, сопоставленный его единственному входу.

Короткий путь для последовательных протоколов

Не реализуйте выборку битов заново

Разобранный пример сам выбирает биты, потому что его протокол самосинхронизирующийся и не похож ни на что встроенное. Для любых протоколов семейства асинхронной последовательной передачи, таких как RS-232, RS-422/485, MODBUS, MIDI, LIN, XBeee и DMX512, не реализуйте выборку битов заново. Вызывайте общую, проверенную на корпусе записей вспомогательную функцию `serial_bytes()`, которая выполняет автоопределение скорости, выборку в центре бита, а также проверки чётности и стоп-бита единственным проверенным способом, и стройте обрамление на потоке байтов, который она возвращает.

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` возвращает четыре значения: байтовые кадры, строку ошибки, пустую при успехе, обнаруженную скорость передачи и число отсчётов на бит. Значение `baud`, равное 0, включает автоопределение скорости. При `inverted`, равном `True`, сигнал читается с обратной полярностью: низкий уровень в покое, активный высокий, как у настоящего RS-232. Протокол более высокого уровня, такой как MODBUS или LIN, вызывает `serial_bytes` для получения необработанных байтов, а затем группирует и проверяет их, формируя собственные кадры; см. `modbus.py` и `lin.py` для этого шаблона.

Установка и тестирование

Установка

Поместите и откройте

Скопируйте готовый файл в доступную для записи папку decoders рядом с tdstoolkit.json и откройте или повторно откройте вкладку «Осциллограммы». Декодировщик появляется в списке протоколов. Ни шага регистрации, ни пересборки нет. Сопоставьте захваченный источник каждой из его ролей, задайте его параметры и запустите его.

ПРИМЕЧАНИЕ. Если декодировщик не появляется, значит, его не удалось импортировать. Проверьте в журнале программы строку с именем файла и исключением. Загрузчик пропускает файл, который не импортируется, не нарушая работу всего списка, поэтому неисправный декодировщик отказывается молча.

Тестирование

Подайте на него
синтетический Signal

Декодировщик — это чистая функция от своих Signal и параметров, поэтому его тестируют без прибора, построив Signal из сформированного массива отсчётов, вызвав decode() и проверив результат с помощью assert. Постройте отсчёты так, чтобы их смысл был известен, оберните их в Signal, вызовите auto_threshold() — единственный шаг, который обычно делает каркас, — и проверьте кадры.

```
from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20          # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20          # idle low tail
    sig = Signal(stream, 1e-6)       # 1 us per sample
    sig.auto_threshold()             # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()
```

Тестируйте каждый декодировщик

Каждый встроенный декодировщик проверяется этим способом. Декодировщик RS-232 прогоняется по корпусу реальных захватов; каждый другой декодировщик проверяется по синтетическому вектору в `tdstest.py`, каждый со своей парной мутацией-«укусом» в `bitecheck.py`, которая намеренно ломает декодировщик и подтверждает, что тест это замечает. Добавляйте тестовый случай для каждого нового протокола.



ВНИМАНИЕ. Непротестированное декодирование со временем незаметно начинает давать сбои. Добавьте синтетический вектор для каждого добавляемого декодировщика и мутацию-«укус», доказывающую, что вектор действительно что-то проверяет, прежде чем полагаться на результат.

Контрольный список

**Прежде чем полагаться
на него**

- Файл назван строчными буквами без пробелов и не начинается с подчёркивания.
- Класс задаёт NAME, DESCRIPTION, SOURCES и PARAMS, и NAME не пустой.
- decode() возвращает кортеж (frames, note) в любой ветви, включая ветви с ошибкой.
- index и end_index кадра абсолютны: sig.first_index плюс локальный индекс отсчёта.
- При нечитаемой записи возвращаются пустой список и примечание с объяснением причины, но никогда не исключение.
- columns(), row() и label() согласованы по числу и порядку полей.
- Синтетический тест подаёт данные на декодировщик и проверяет декодированные значения с помощью assert.